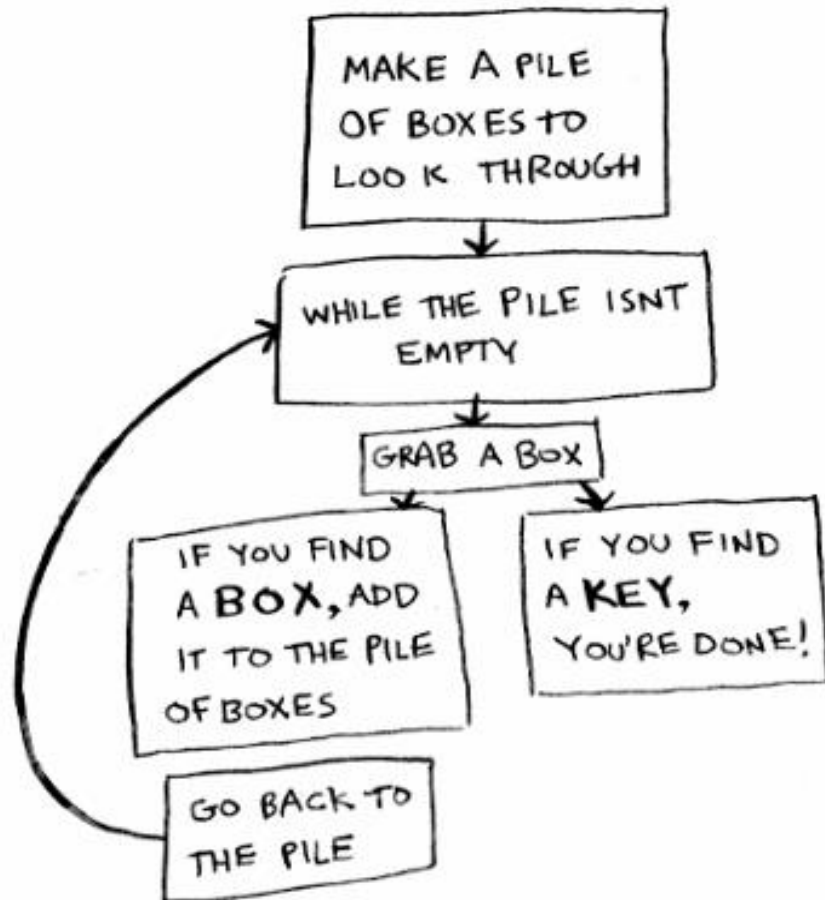


Algoritmos y Estructuras de Datos

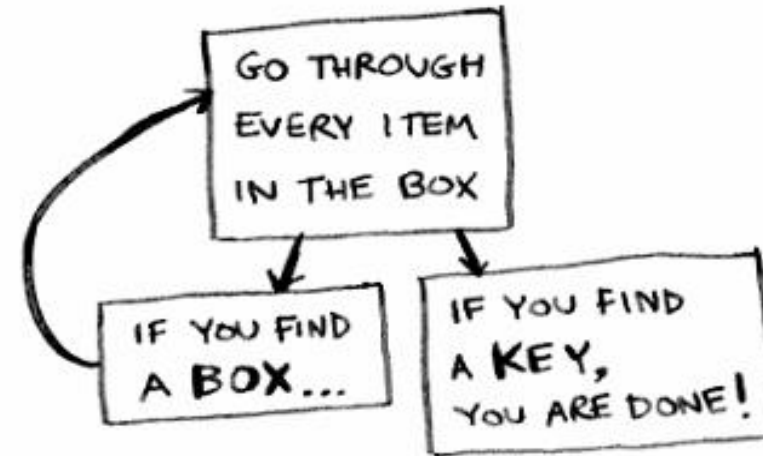
1er semestre - 2026

Recursión

Iterative Approach



Recursive Approach



Fuente: <https://www.freecodecamp.org/news/how-recursion-works-explained-with-flowcharts-and-a-video-de61f40cb7f9>

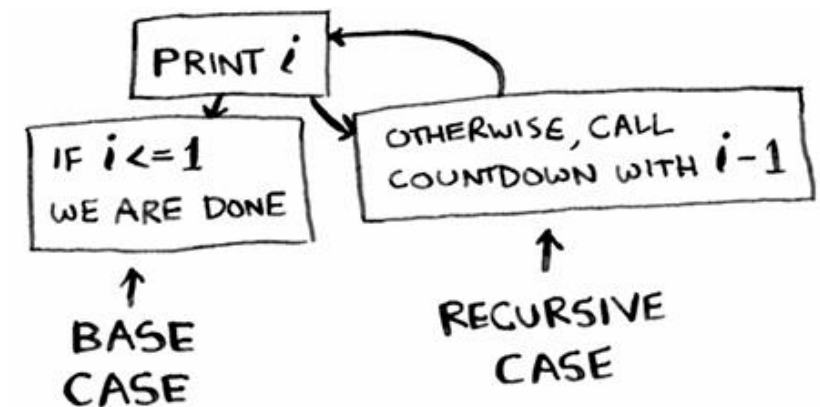
Recursión

Definición

- Un proceso que se define a partir de sí mismo
- Un método que *se llama a sí mismo*

¿Cómo funciona?

- Caso base → resuelve lo más simple
- Paso recursivo → divide el problema en instancias más pequeñas y aplica la misma idea



Recursión

Un objeto es recursivo si:

- En parte está formado por sí mismo, o
- Está definido en función de sí mismo

Ejemplos en matemáticas

- Número natural:
 - 0 pertenece a $\mathbb{N}$
 - Si n pertenece a $\mathbb{N}$, entonces $n+1$ pertenece a $\mathbb{N}$
- Factorial, potencia
- Números de Fibonacci

Ejemplo: Factorial

Definición recursiva

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

Ejemplo: Factorial

Definición recursiva

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n - 1) & \text{else} \end{cases}$$

Como método

```
// función factorial recursiva
public static int factorial(int n){
    if (n == 0) return 1; // caso base
    return n * factorial(n-1); // caso recursive
}
```

Recursión

Una función es recursiva cuando se invoca a sí misma,

- Es útil cuando el problema a resolver o la estructura de datos a recorrer son de naturaleza recursiva.
- Evaluar muy bien para otros casos, y evitar su uso si existe una solución no recursiva aceptable



Recursión

- Los algoritmos recursivos son idóneos principalmente cuando está definido en forma recursiva:
 - El problema a resolver, o
 - La función por calcular, o
 - La estructura de datos por procesar
- Permite definir un conjunto infinito de objetos mediante una proposición finita.
- Un número infinito de cálculos puede describirse mediante un programa recursivo finito, sin repeticiones explícitas.

Recursión

- Si un módulo **P** contiene una referencia explícita a sí mismo, se dice que es **directamente recursivo**.
 - Técnica ampliamente usada para la resolución de problemas.
- Si un módulo **P** contiene una referencia a un módulo **Q** que a su vez referencia a **P**, entonces se dice que **P** es **indirectamente recursivo**.
 - Técnica de **cuidadosa aplicación**, o error que debe ser evitado.

Debe tenerse en cuenta el problema de la terminación; por ejemplo, si un módulo **P** de tamaño “n” consiste de una secuencia “S” de sentencias:

$$P(n) \equiv P[S, \text{Si } n > 0 \Rightarrow P(n-1)]$$

Uso de la recursión – una forma usual

Función A (N: tamaño del problema)

COM

<bloque>

Si <condición> entonces

<bloque>

A (N-1)

Sino

<bloque>

Fin Si

<bloque>

FIN

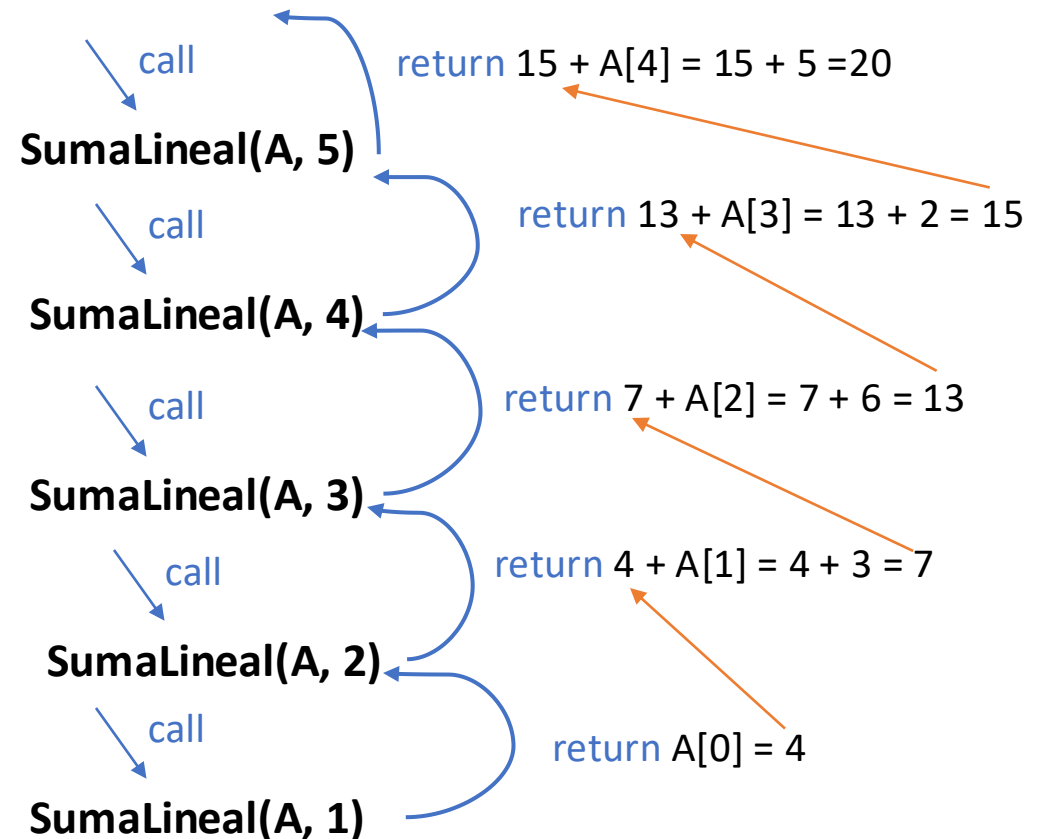
Recursión Lineal

- Probar los casos base.
 - **Comenzar** probando un conjunto de casos base (debe haber **por lo menos uno**).
 - Cada cadena de llamadas recursivas potencial **debe** eventualmente alcanzar un caso base, y el manejo del caso base **NO DEBE USAR RECURSIVIDAD**
- Recurrir una vez.
 - Realizar una sola llamada recursiva. (este paso puede involucrar una prueba para decidir cuál de varias posibles llamadas recursivas diferentes ejecutar, pero en última instancia **solo se ha de ejecutar una**)
 - Definir cada posible llamada recursiva de forma tal que **progrese hacia un caso base**.

Ejemplo simple de recursión lineal

Ejemplo de secuencia recursiva:
 $A = \{4, 3, 6, 2, 5\}$ y $n = 5$

```
procedure SUMALINEAL(A, n)
  // A es un arreglo de enteros
  //  $n \geq 1$ 
  // A tiene al menos  $n$  elementos
  // suma de los primero  $n$  enteros en A
  if  $n == 1$  then
    return  $A[0]$ 
  else
     $actual \leftarrow A[n - 1]$ 
     $rec \leftarrow sumaLineal(A, n - 1)$ 
    return  $rec + actual$ 
  end if
end procedure
```



Recursión

- Soluciones a funciones matemáticas recursivas: **escribir un algoritmo en pseudocódigo para:**
 - Hallar el factorial de un número natural “N”.
 - Hallar la potencia de un número “N” elevado a un exponente “e”.
 - Hallar el número de Fibonacci correspondiente a la posición “N” de la sucesión

RECORDAR Ejemplo: Factorial

Definición recursiva

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot f(n - 1) & \text{else} \end{cases}$$

Como método

```
// función factorial recursiva
public static int factorial(int n){
    if (n == 0) return 1; // caso base
    return n * factorial(n-1); // caso recursive
}
```

Cálculo de la potencia

- La función potencia, $p(x, n) = x^n$, puede ser definida recursivamente

$$p(x, n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot p(x, n - 1) & \text{else} \end{cases}$$

- Esto conduce a una función potencia que se ejecuta en un tiempo $O(n)$ (puesto que hacemos n llamadas recursivas)
- Desarrollar un pseudocódigo !!

Recursión Binaria

- Ocurre cada vez que hay **dos llamadas recursivas** para cada caso no-base
- Ejemplo: Números de Fibonacci
- Los números de Fibonacci se definen recursivamente:

$$f(x) = \begin{cases} 0 & \text{if } x = 0 \\ 1 & \text{if } x = 1 \\ f(x - 1) + f(x - 2) & \text{else} \end{cases}$$

Algoritmo recursivo (primera versión)

Algoritmo **FibonacciBinario(k)**

Entrada: entero k no negativo

Salida: el k-esimo número de Fibonacci

COM

SI $k = 0$ o $k = 1$

 devolver k

SINO

 devolver **FibonacciBinario(k - 1) + FibonacciBinario(k - 2)**

FIN SI

FIN

Análisis del Algoritmo Recursivo Binario de Fibonacci

n_k indica el número de llamadas recursivas hechas por `FibonacciBinario(k)`. Entonces

- $n_0 = 1$
- $n_1 = 1$
- $n_2 = n_1 + n_0 + 1 = 1 + 1 + 1 = 3$
- $n_3 = n_2 + n_1 + 1 = 3 + 1 + 1 = 5$
- $n_4 = n_3 + n_2 + 1 = 5 + 3 + 1 = 9$
- $n_5 = n_4 + n_3 + 1 = 9 + 5 + 1 = 15$
- $n_6 = n_5 + n_4 + 1 = 15 + 9 + 1 = 25$
- $n_7 = n_6 + n_5 + 1 = 25 + 15 + 1 = 41$
- $n_8 = n_7 + n_6 + 1 = 41 + 25 + 1 = 67$

Algoritmo *FibonacciBinario(k)*

COM

SI $k=0$ o $k=1$

devolver k

SINO

devolver

***FibonacciBinario(k - 1) +
FibonacciBinario(k - 2)***

FIN SI

FIN

Un mejor algoritmo de Fibonacci - utilizando recursión lineal

Algoritmo FibonacciLineal (k)

Entrada: entero k no negativo

Salida: par de números de Fibonacci (F_k, F_{k-1})

COM

SI k = 1

 devolver (k, 0)

SINO

 (i, j) **FibonacciLineal**(k - 1)

devolver (i+j, i)

Fin si

FIN

- Ejecuta en tiempo $O(k)$

Ejercicios

- Implementar en Java
 - Fibonacci Recursivo
 - Imprimir lista al revés
 - Invertir lista

Link sobre recursividad

- <http://es.wikipedia.org/wiki/Recursividad>
- http://es.wikipedia.org/wiki/Algoritmo_recursivo
- <http://es.wikipedia.org/wiki/Factorial>
- http://es.wikipedia.org/wiki/Sucesion_de_Fibonacci
- http://es.wikipedia.org/wiki/Torres_de_Hanoi

Animación de algoritmos:

- <http://users.encs.concordia.ca/~twang/WangApr01/RootWang.html>
- <http://www.mazeworks.com/hanoi/index.htm>
- [http://www.raptivity.com/DemoCourses/Interactivity Builder Sample Course/Content/Booster Pack/HTML Pages/Towers of Hanoi.htm](http://www.raptivity.com/DemoCourses/Interactivity_Builder_Sample_Course/Content/Booster_Pack/HTML_Pages/Towers_of_Hanoi.htm)